Graph Algorithms

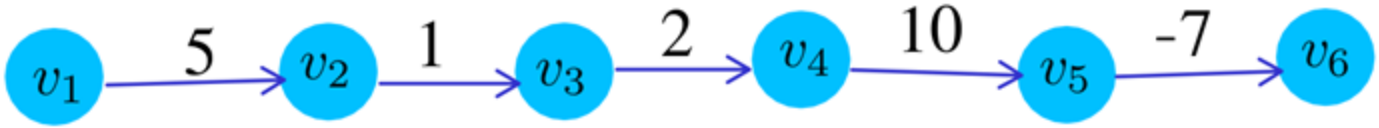
Shortest Path

- Chapter 24 of CLR
  - Shortes-path
  - Dijkstra
  - Bellman-ford

Shortest path

- Consider a digraph G = (V, E) with edge-weight function w : E → R. The weight of path P = (v_1, v_2, …, v_k) is defined to be

$$w(P) = \sum_{i=1}^{k-1} w(v_i, v_{i+1})$$



- A **shortest path** from u to v is a path of minimum weight from u to v.
- **δ(u, v)** = min { w(P): P is a path from u to v}
- δ(u, v) = ∞ if no path from u to v exists.

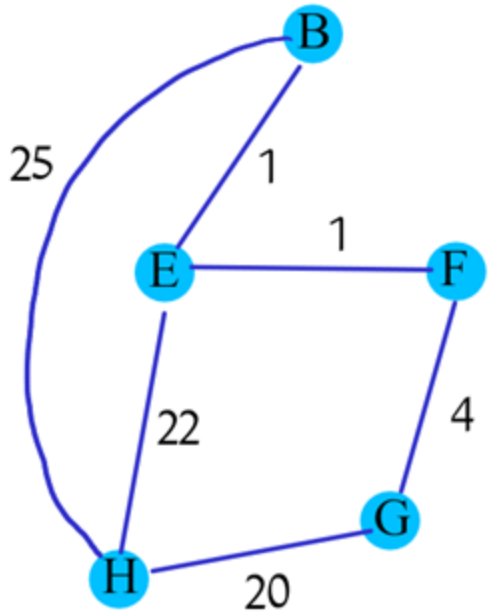
Finding Shortest paths in graphs: single-source shortest path

- **Input**: a graph (directed/undirected) G=(V, E) with **non-negative** edge weights(w(e) ≥ 0), and a starting node **s**
- **Output:** A shortest path from s to each vertex in the graph
- Single-Source-Shortest-Path problem
- The length of the shortest path and then find the shortest path

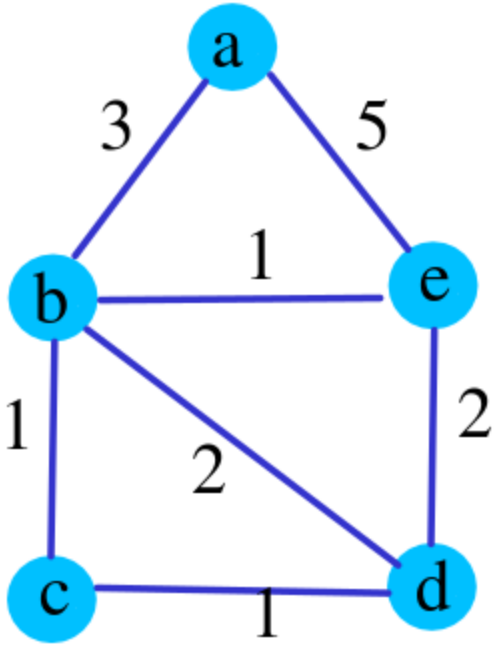


Why BFS is not enough for finding the shortest path?

What is the shortest path from B to G?



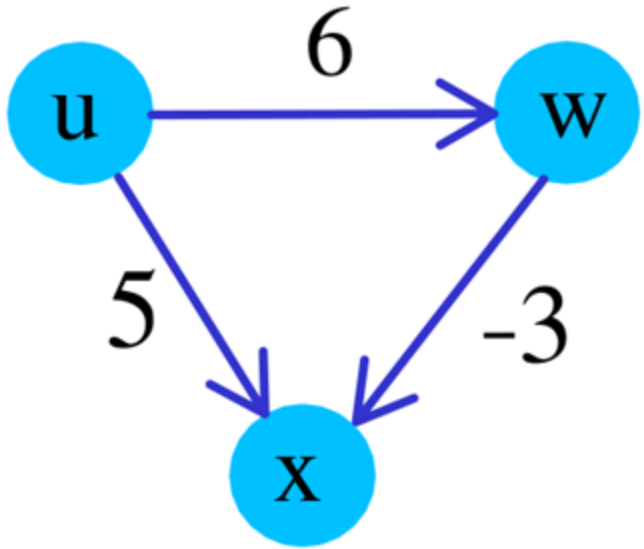
Why MST algorithms are not enough?



negative-weights edges

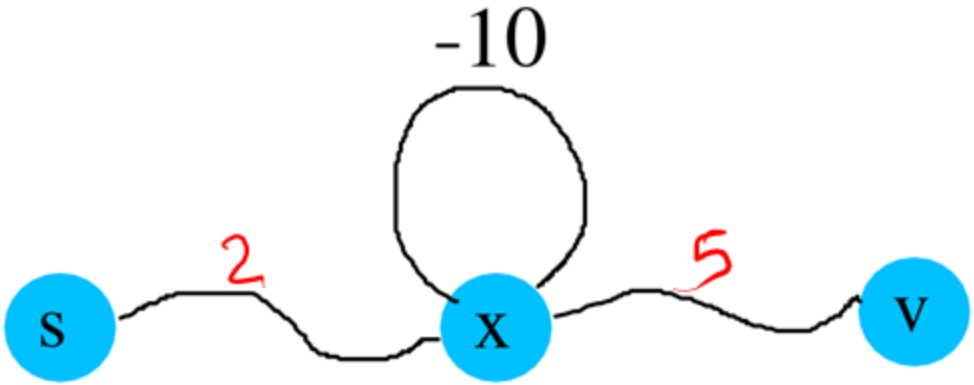
What does it mean to have negative-weight edge?

Negative edge weights mean benefit.



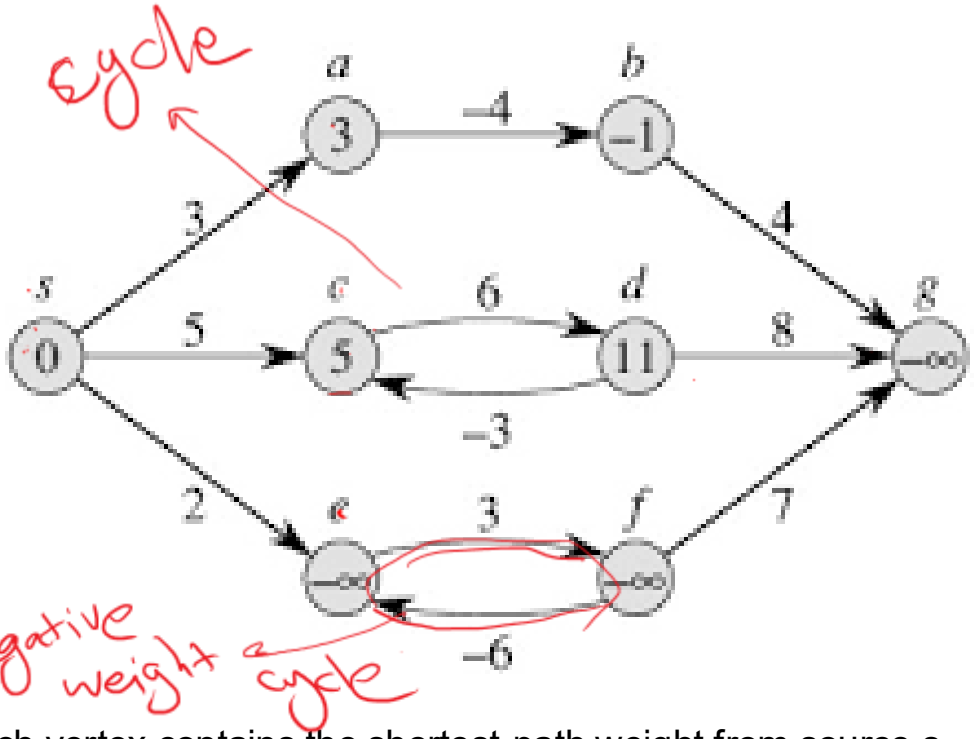
Negative-weights **cycles**

- If the graph G contains a negative-weight cycle reachable from s
  - We can go around the negative cycle as many times as we want
  - shortest-path weights are not well defined
- If G contains no negative-weight cycle reachable from the source s
  - for all v ∈ V, the shortest-path weight is well defined (it could have a negative value)

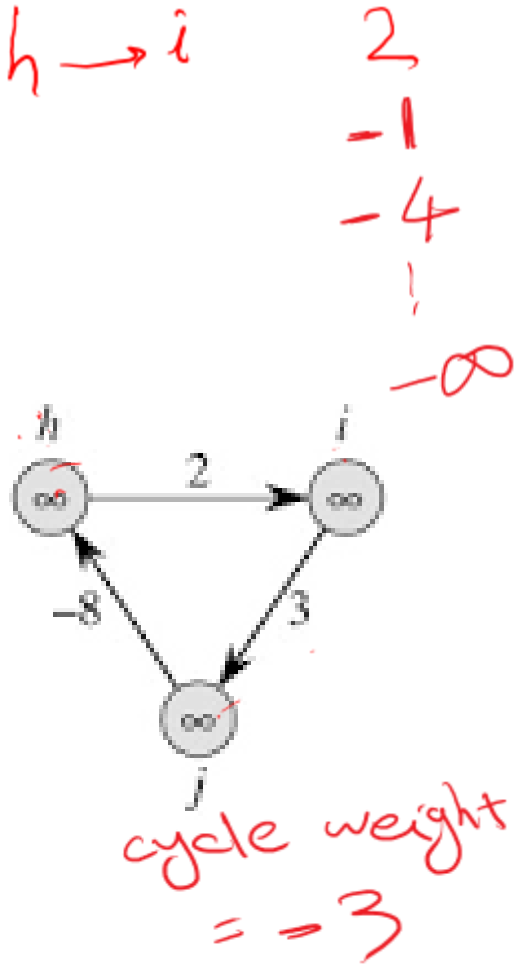


s - x - y → 7
→ -3
-13
-23
⋮
-∞

Negative-weights edges and cycles



Each vertex contains the shortest-path weight from source s.



No cycles in a shortest path

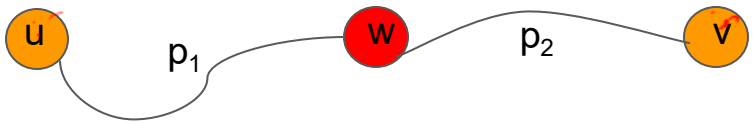
Can a shortest path contain a cycle?

- negative-weight cycle → NO
- positive-weight cycle → NO
- 0-weight cycle → NO

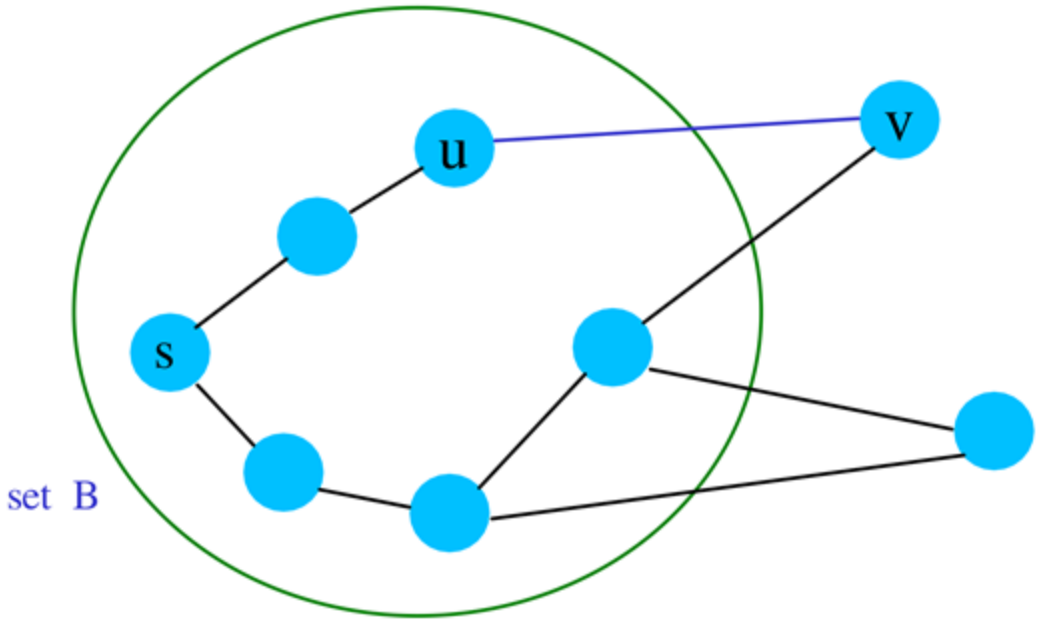
  Shortest paths are simple: can contains at most |v| distinct vertices and at most |V|-1 edges

Shortest path: Optimal Substructure property

- **Optimal Substructure property:**
  - **Optimal solution to the problem contains optimal solution to the subproblems**
- **Example: Shortest path in graphs**
  - **P**: the shortest path between u and v.
  - **Claim**: p_1 is a shortest path from u to w
    - If there were another path, say p'_1 from u to w with less weight, we could cut out p_1 and paste in p'_1 to produce a path $p' = p'_1 + p_2$ with fewer edges → contradiction: p_1 is an optimal solution or the shortest path
    - Similarly we can show p_2 is the shortest path from w to v



Shortest path: Greedy Algorithm: Dijkstra's Algorithm: Idea



Greedy choice: add the vertex with the minimum distance from s

Shortest path: Greedy Algorithm: Dijkstra's Algorithm: Idea

dist[v] = ∞ for all v ∈ V

dist[s] = 0

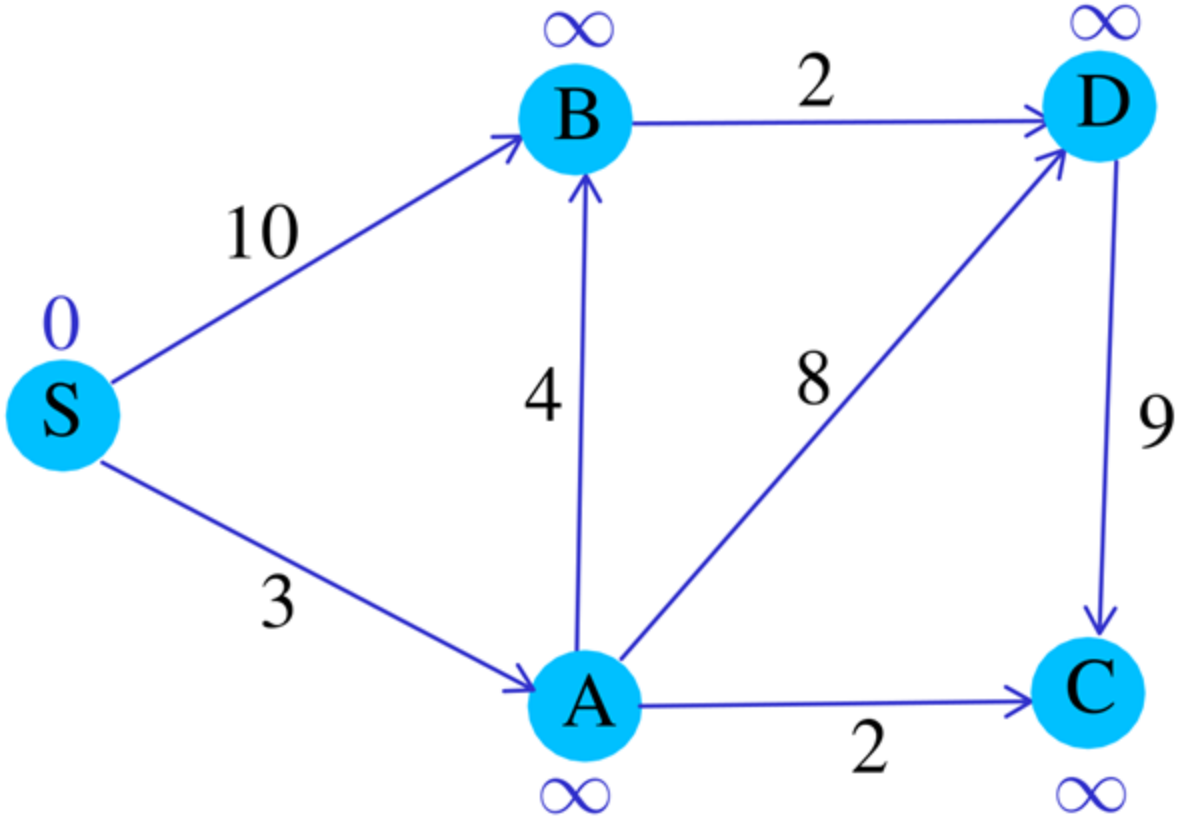
B = ∅ # B is a set of vertices with known shortest distance to s

While B ≠ V

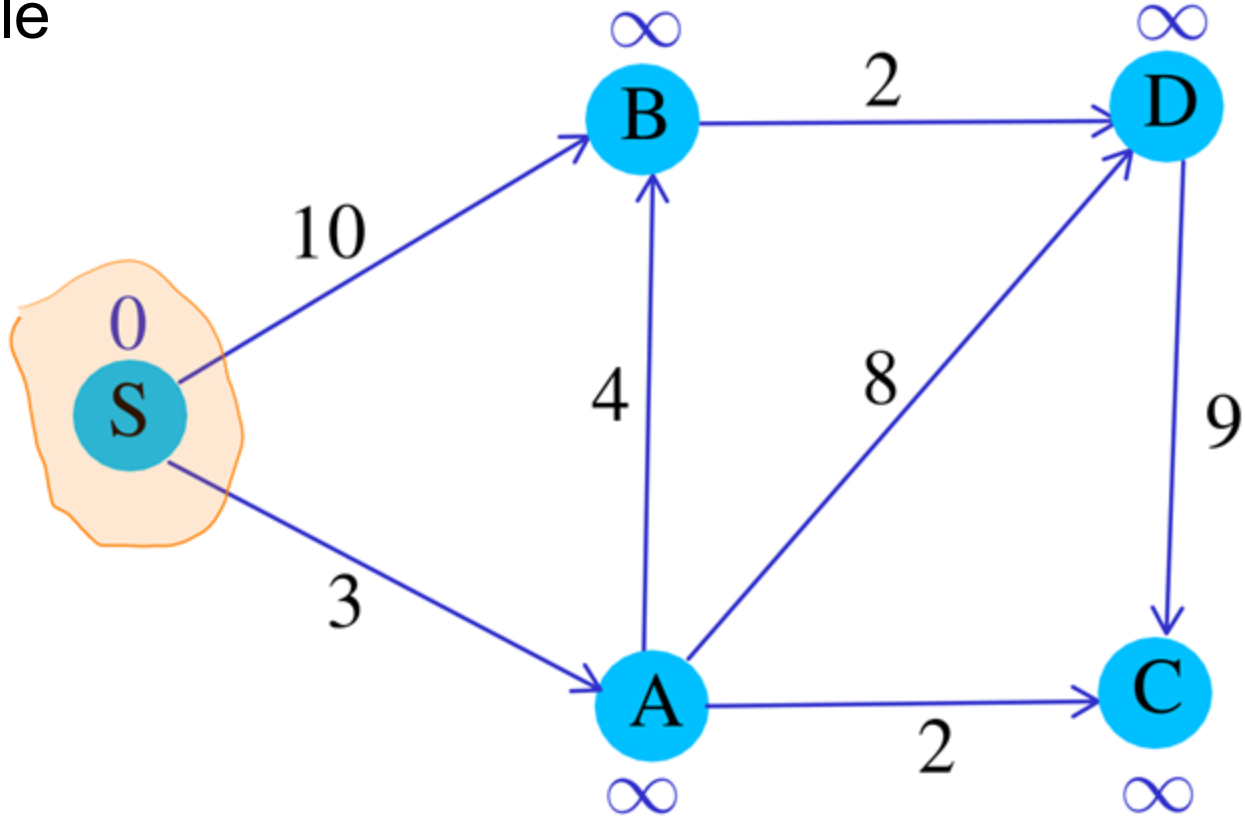
Choose edge (u, v), u ∈ B, v ∉ B to minimize d(s, u) + w(u, v)

Update d[v]: distance of S to v

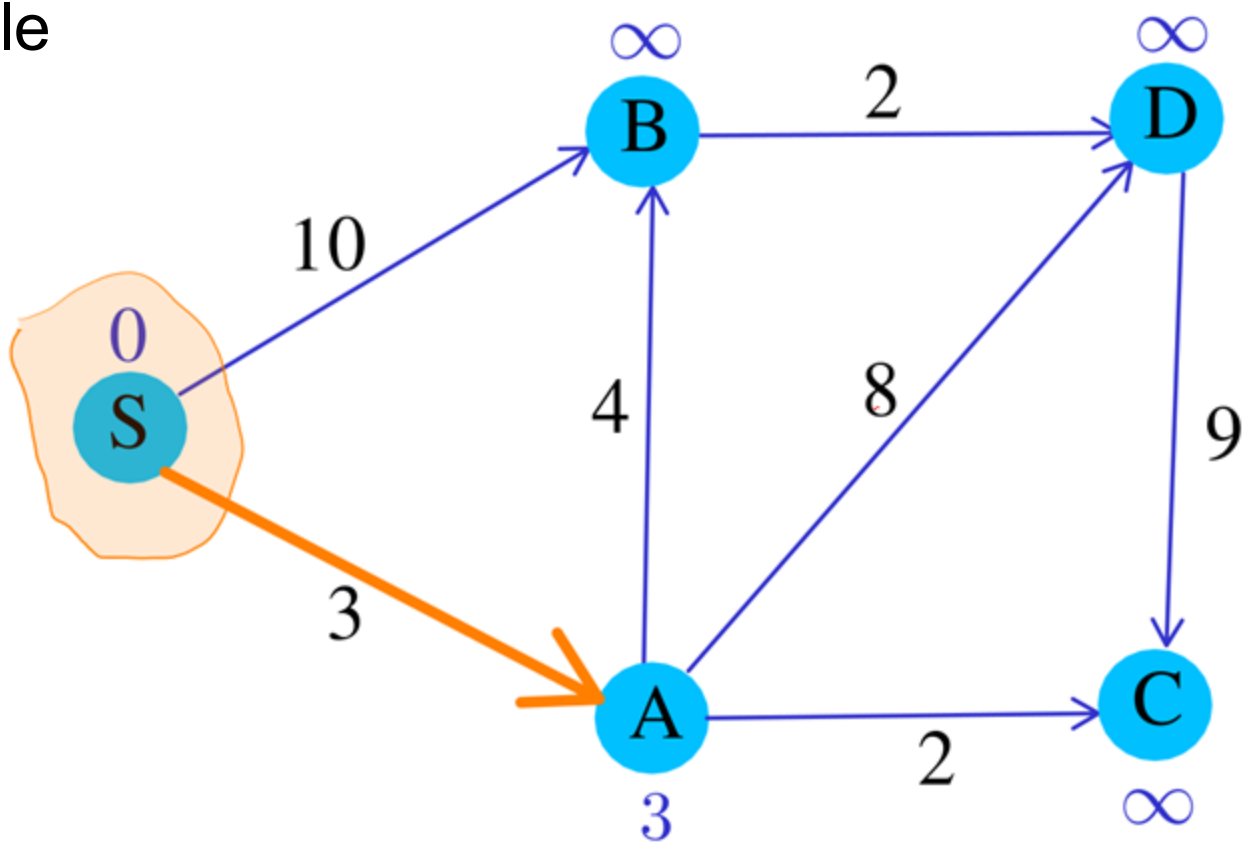
Example



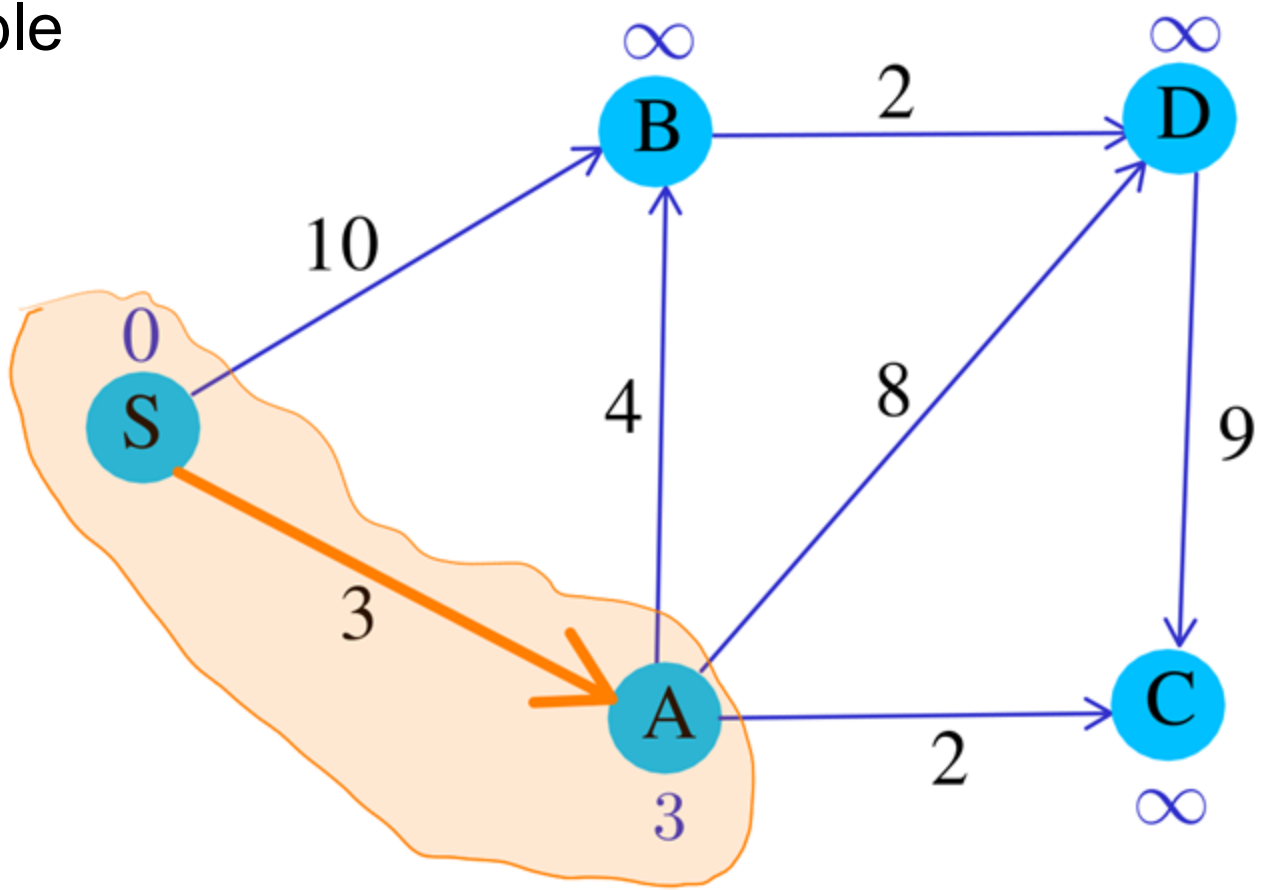
Example



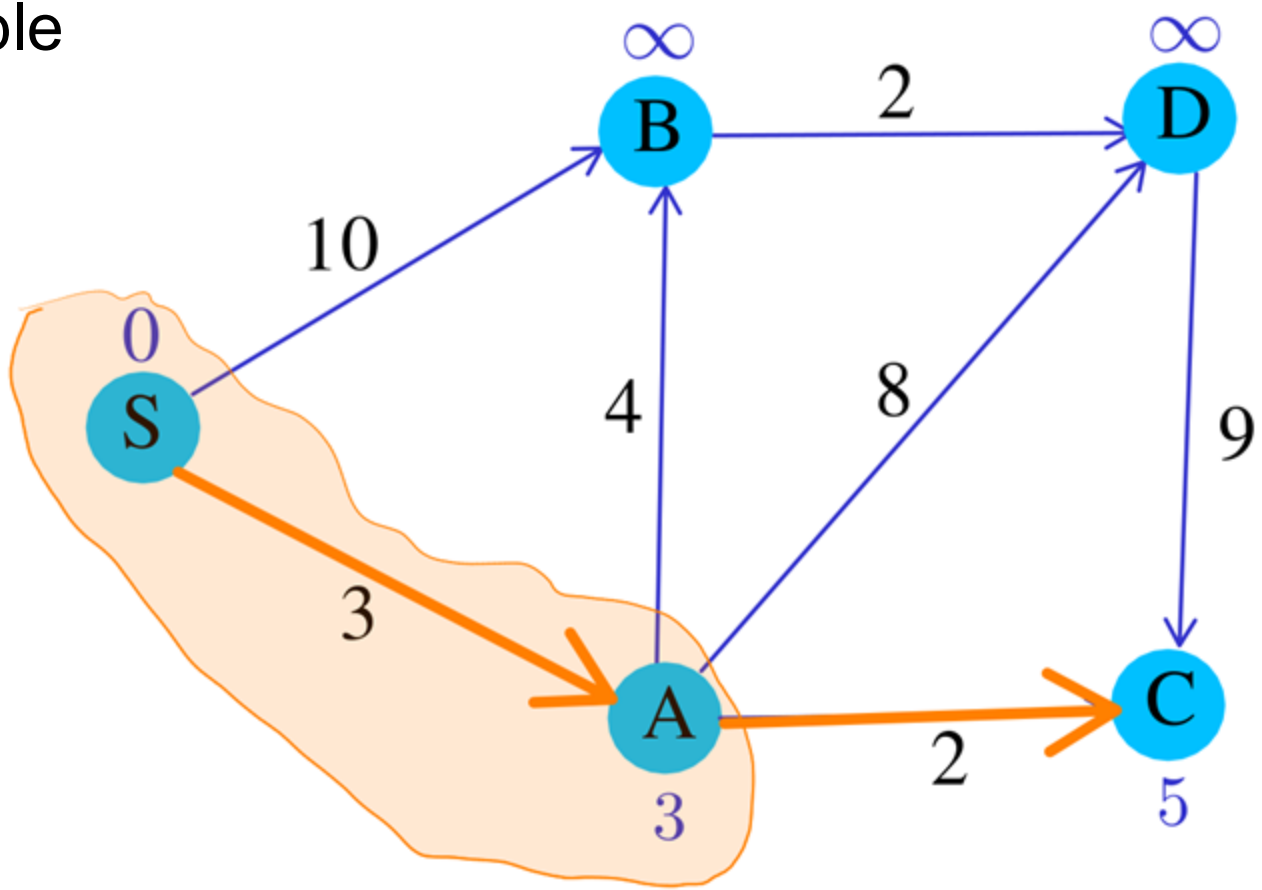
Example



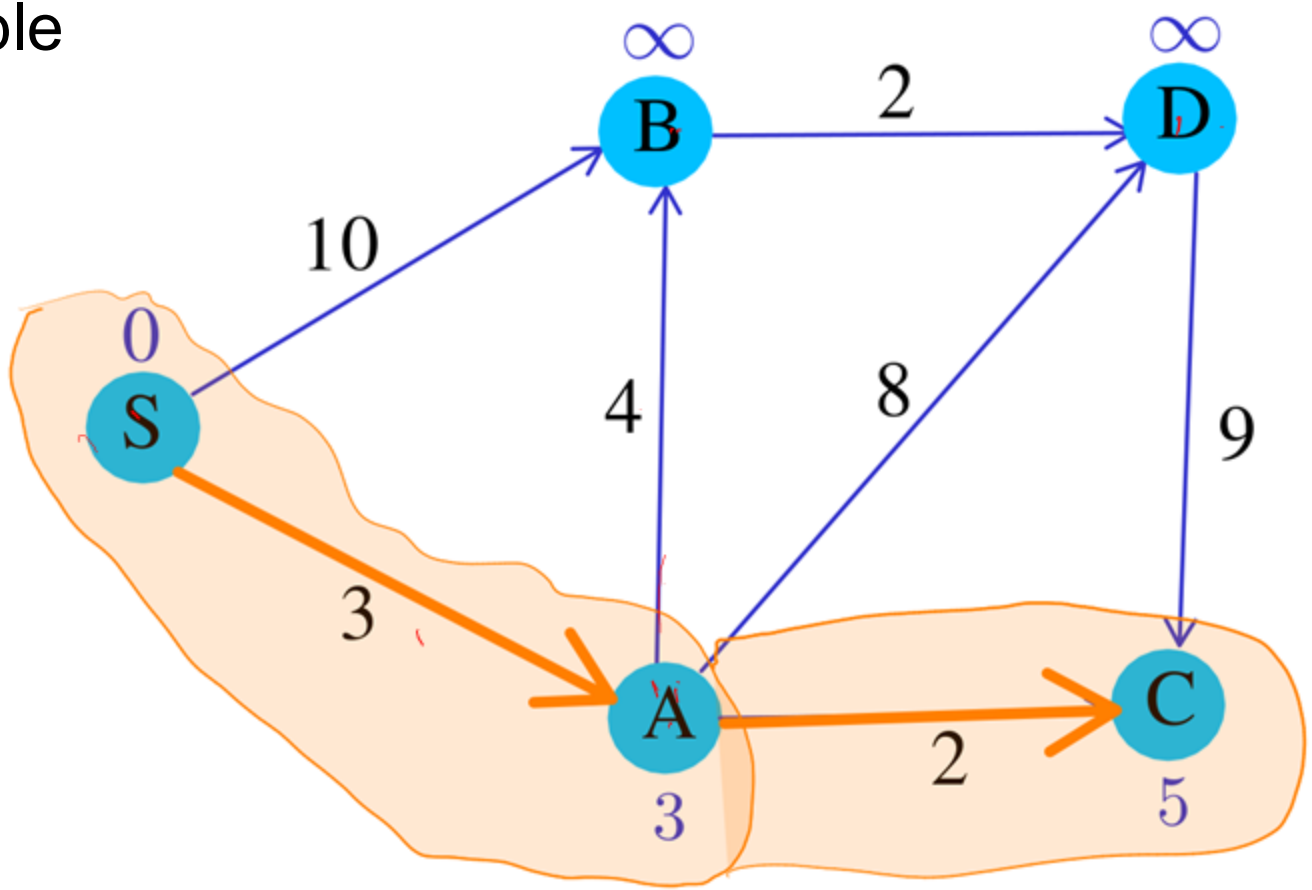
Example



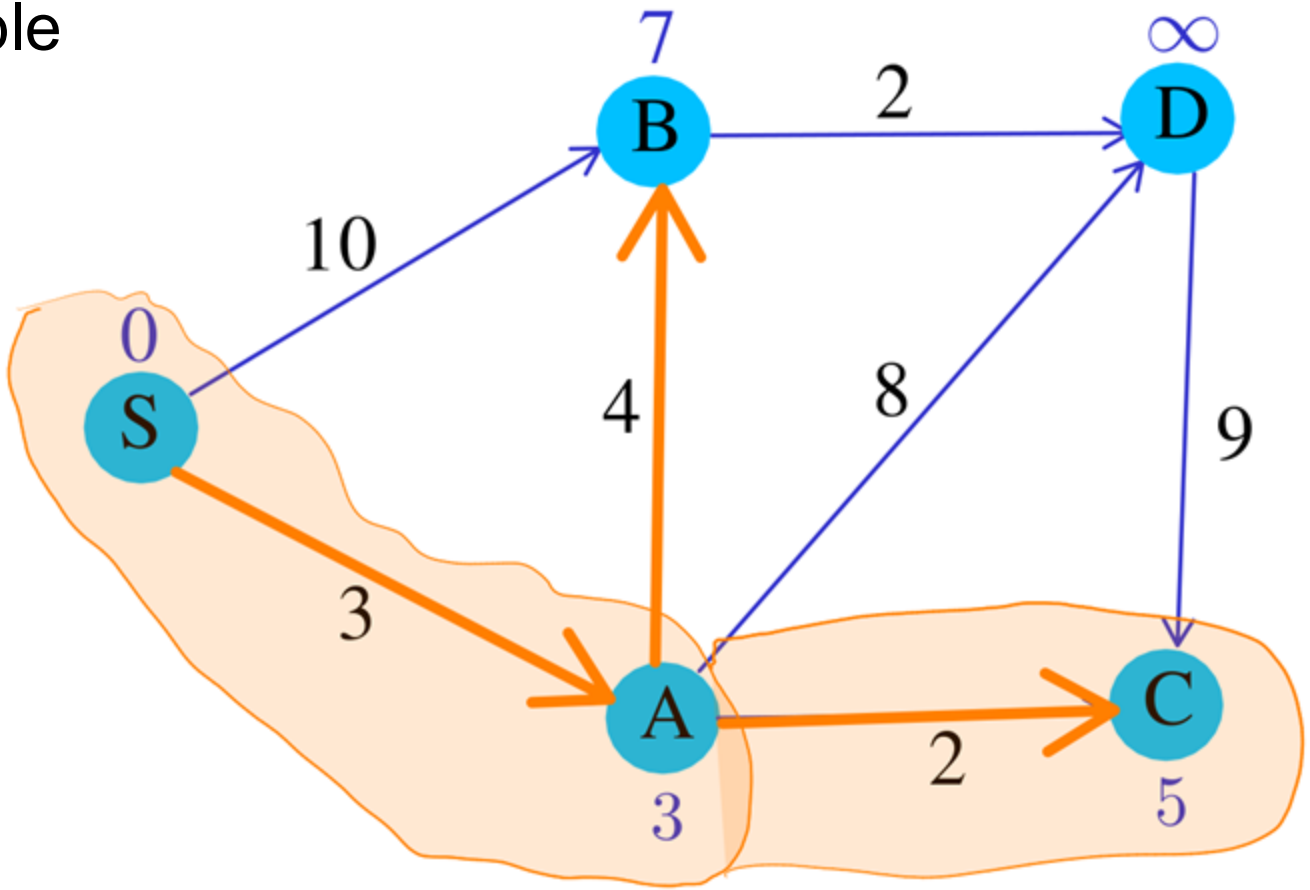
Example



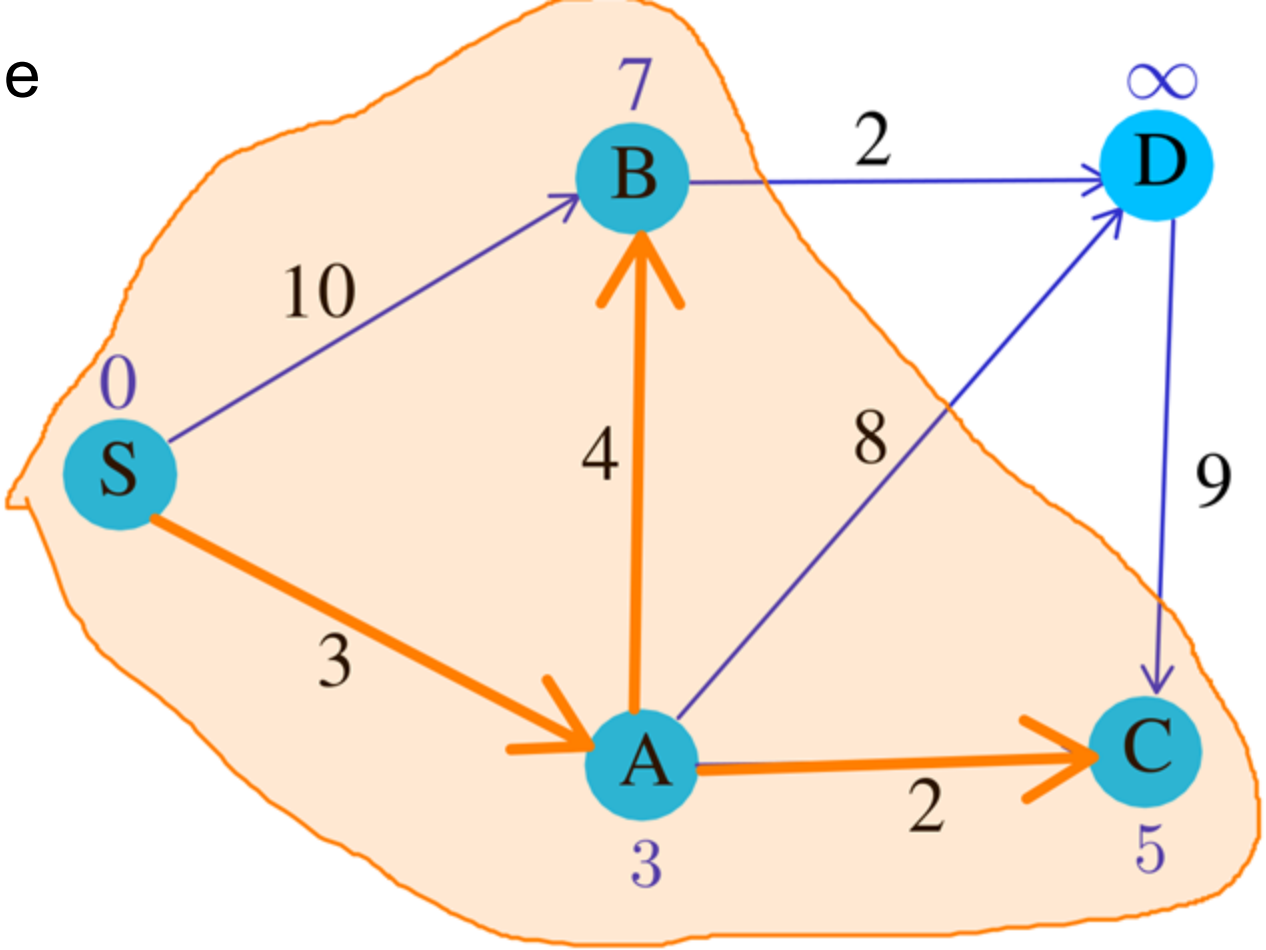
Example



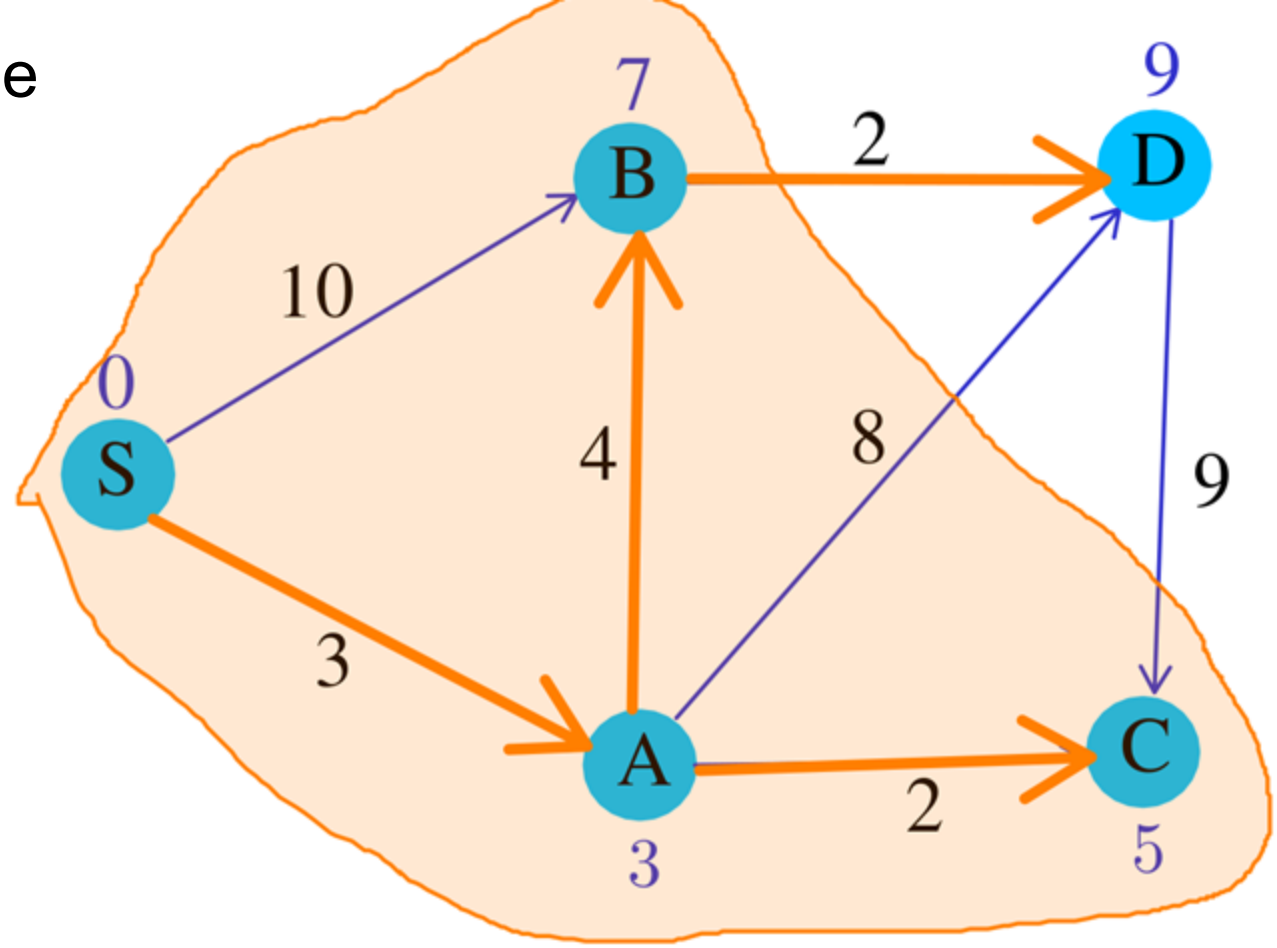
Example



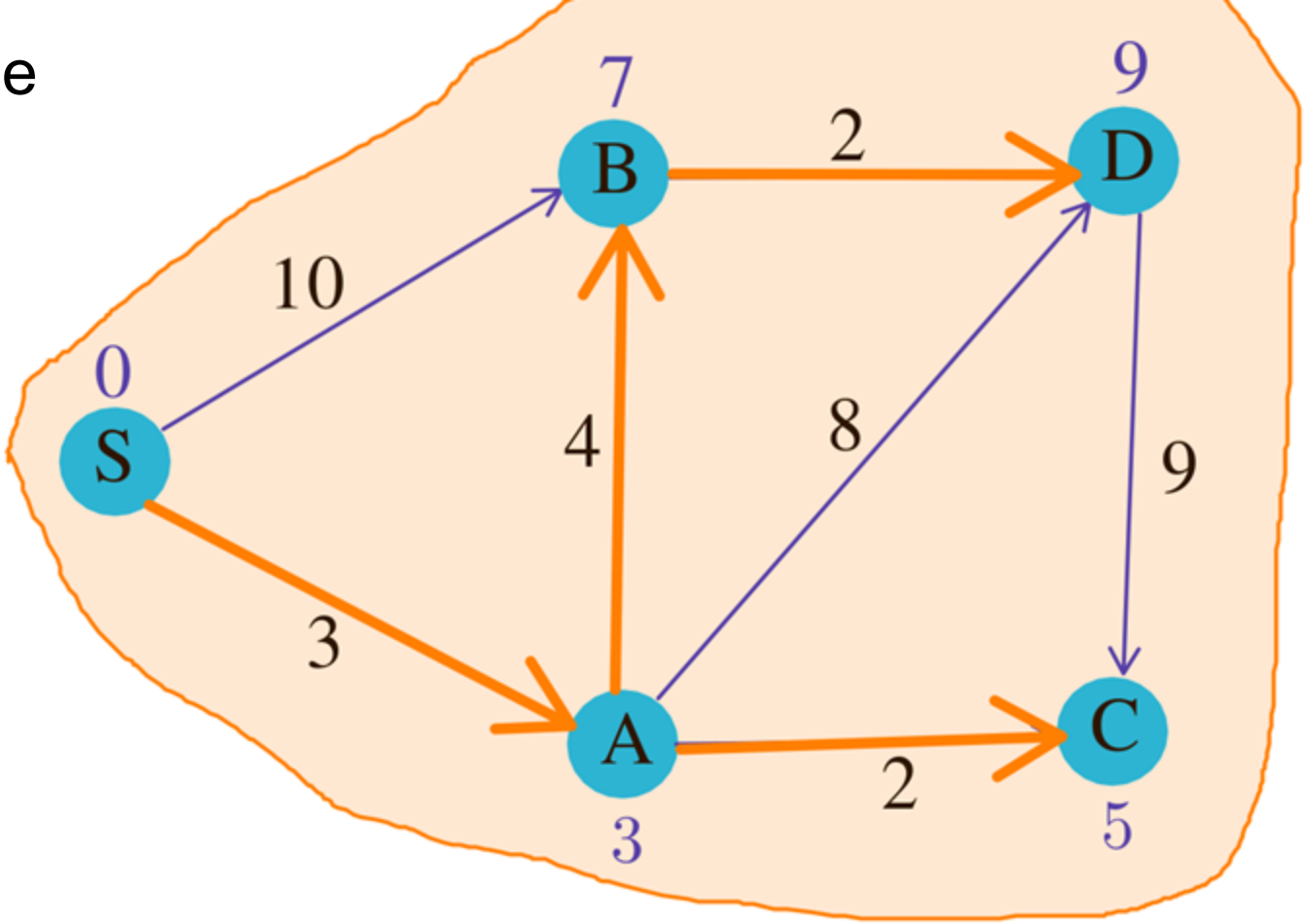
Example



Example



Example

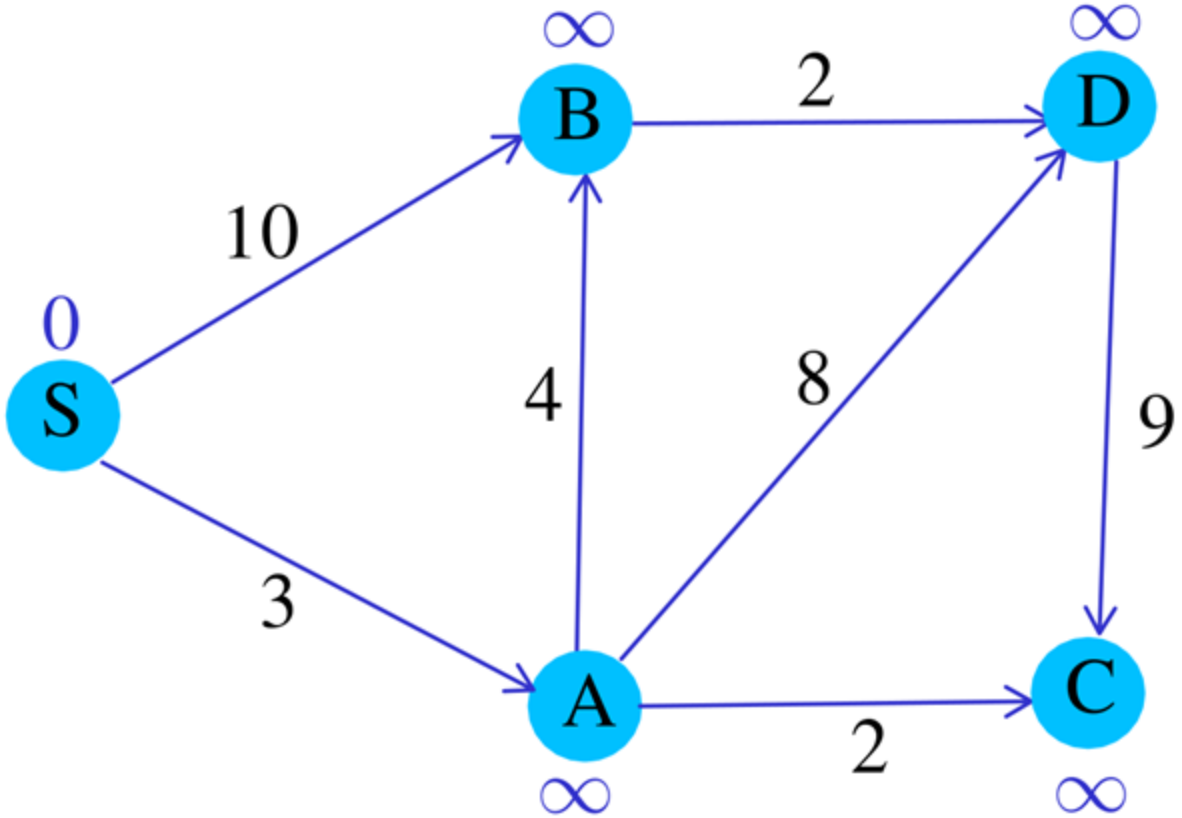


Add the relaxation step to have a more efficient

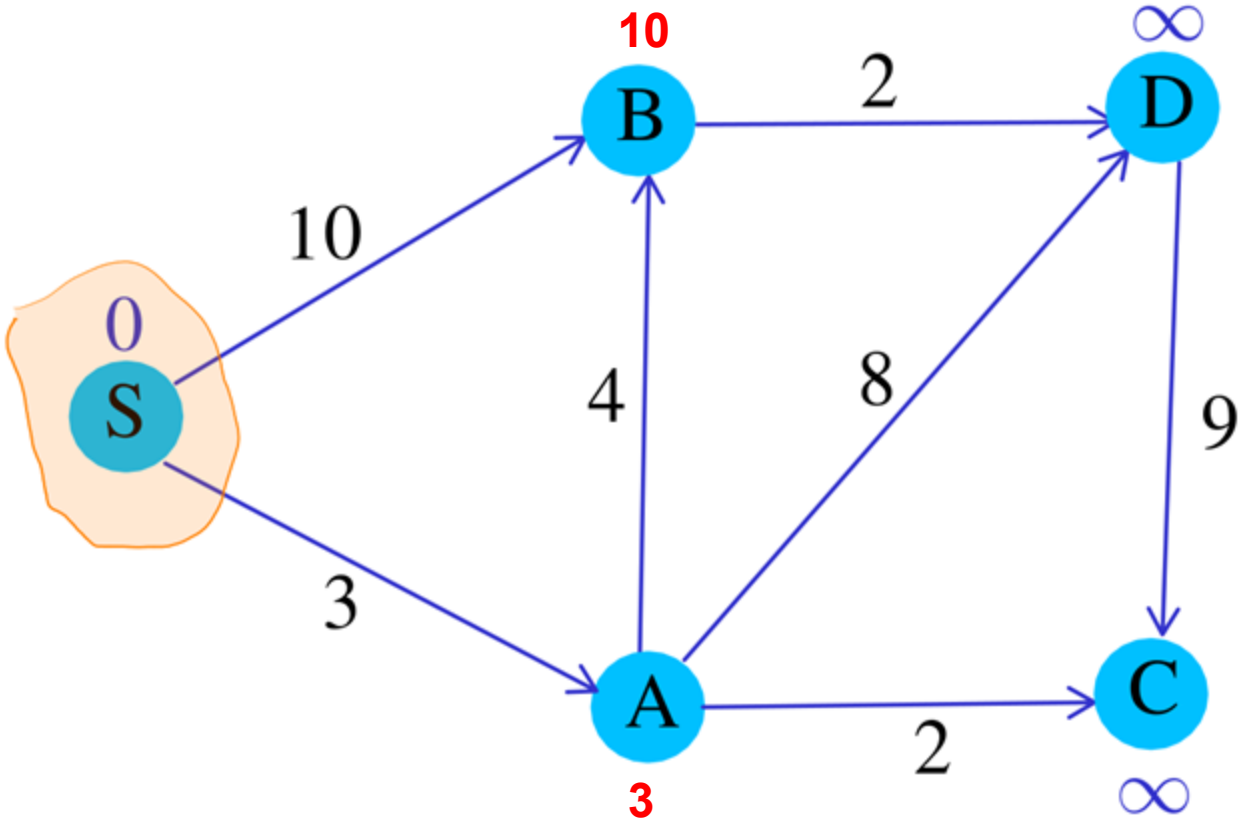
Whenever a node, u, is added to B, update the distance from s to all neighbors of u which are not in B

To pick a node to add to B, pick the node with minimum distance

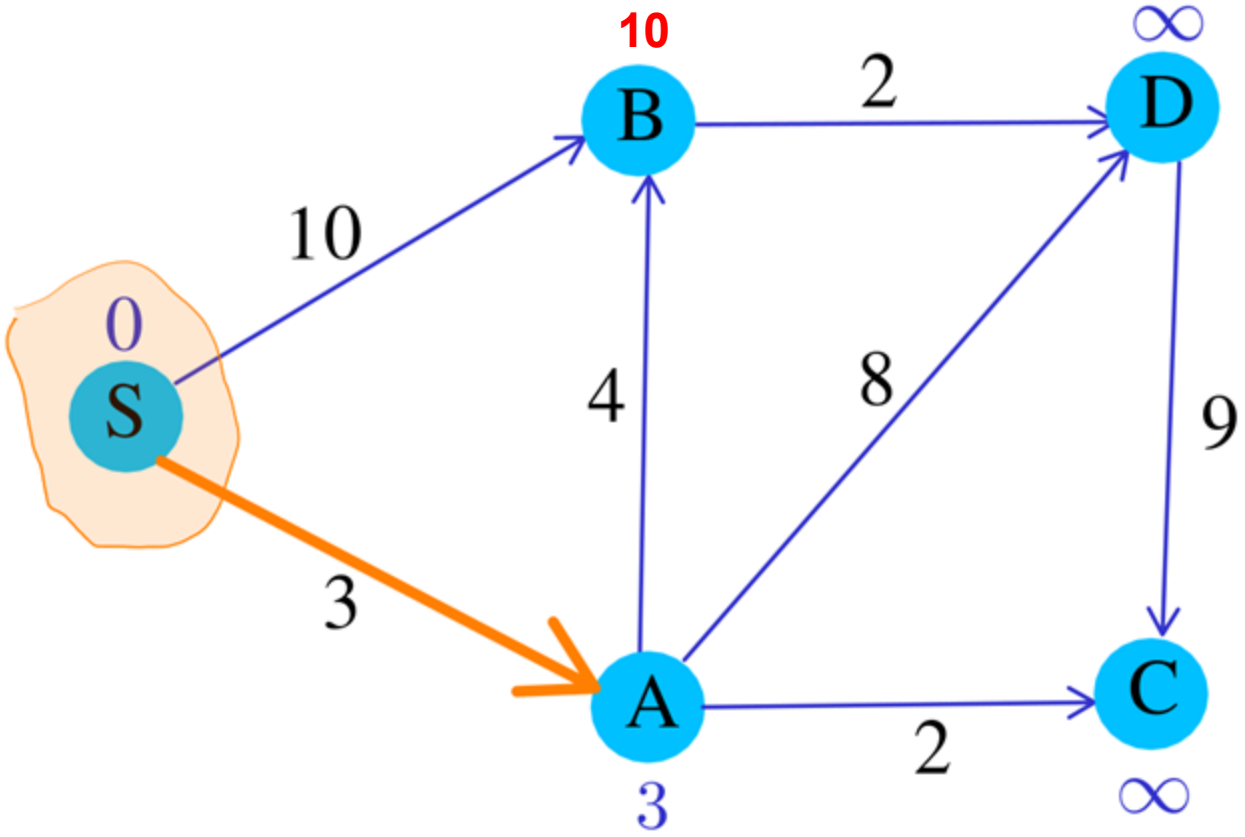
Example



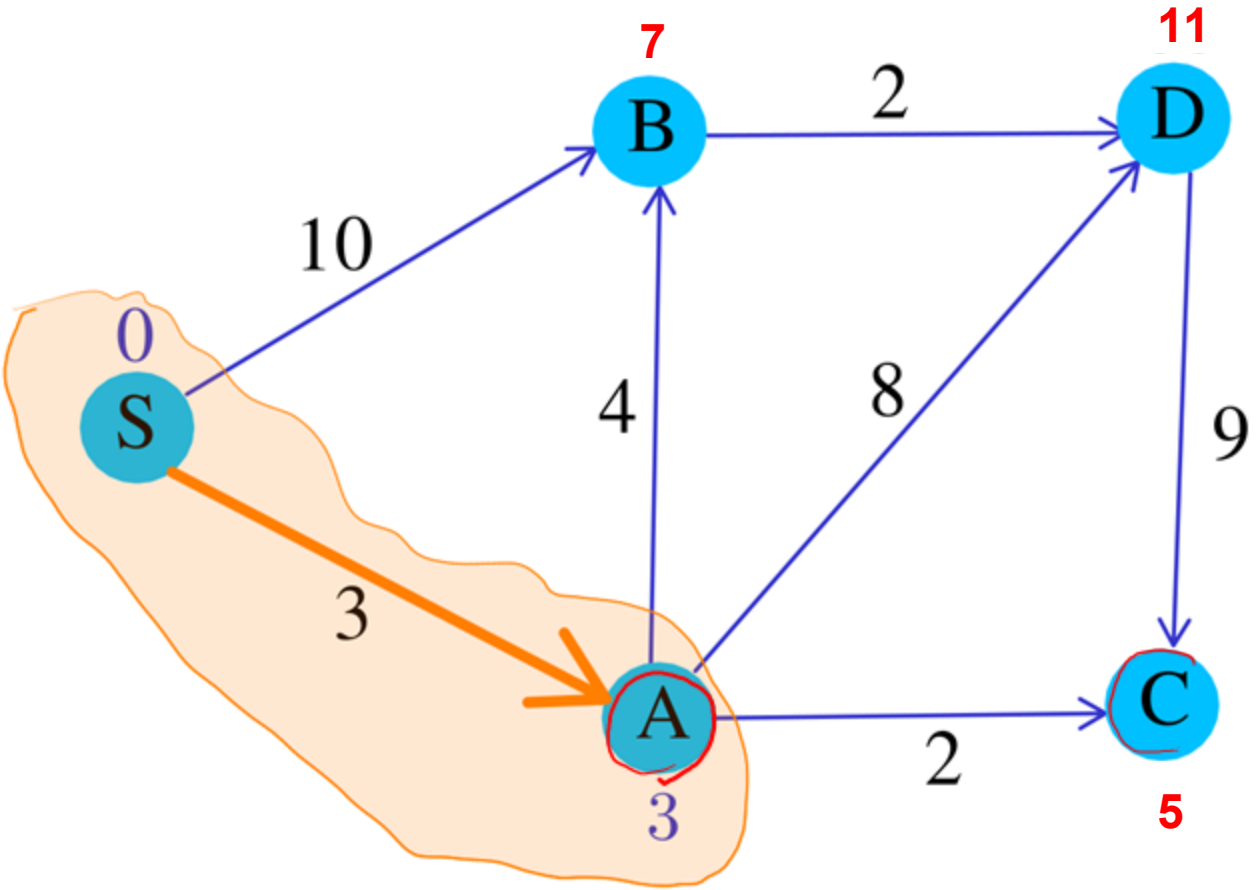
Example



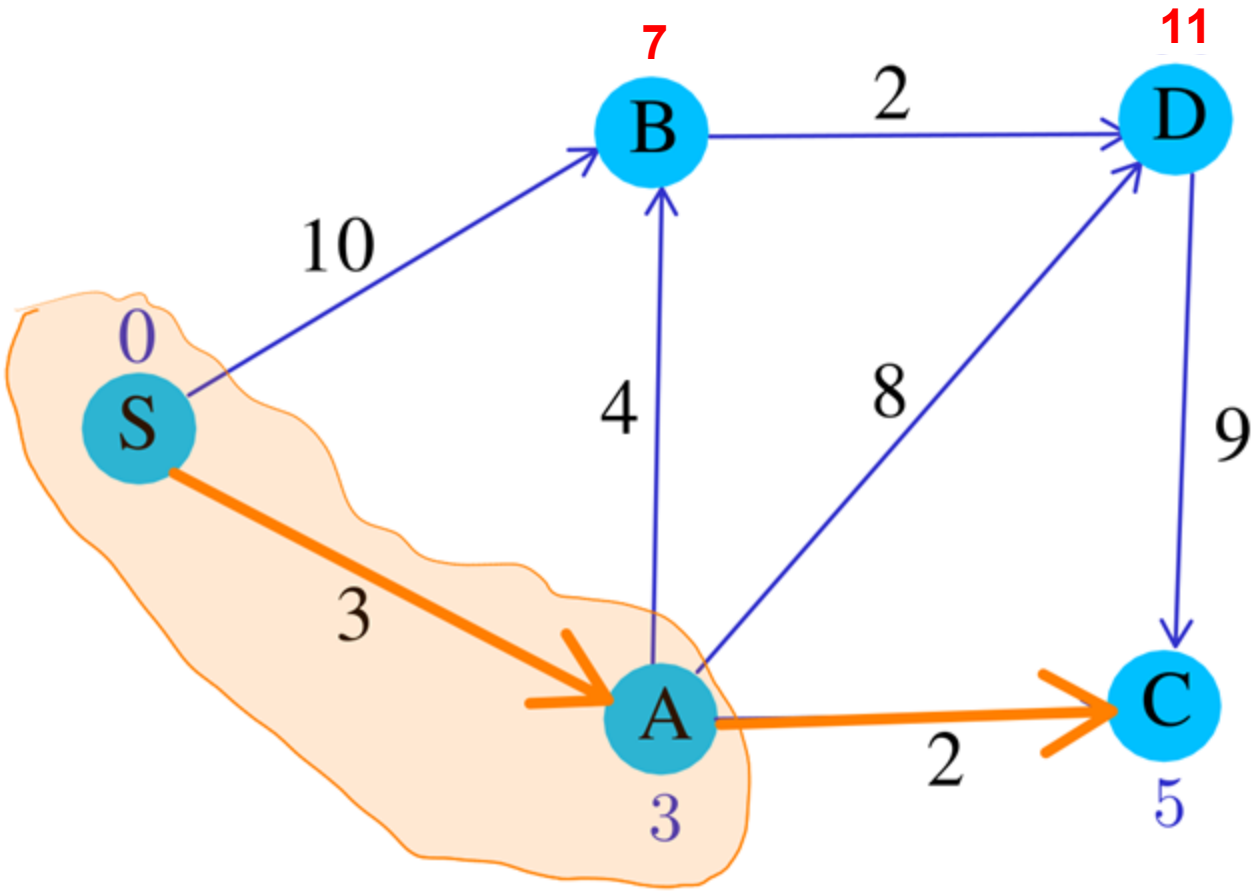
Example



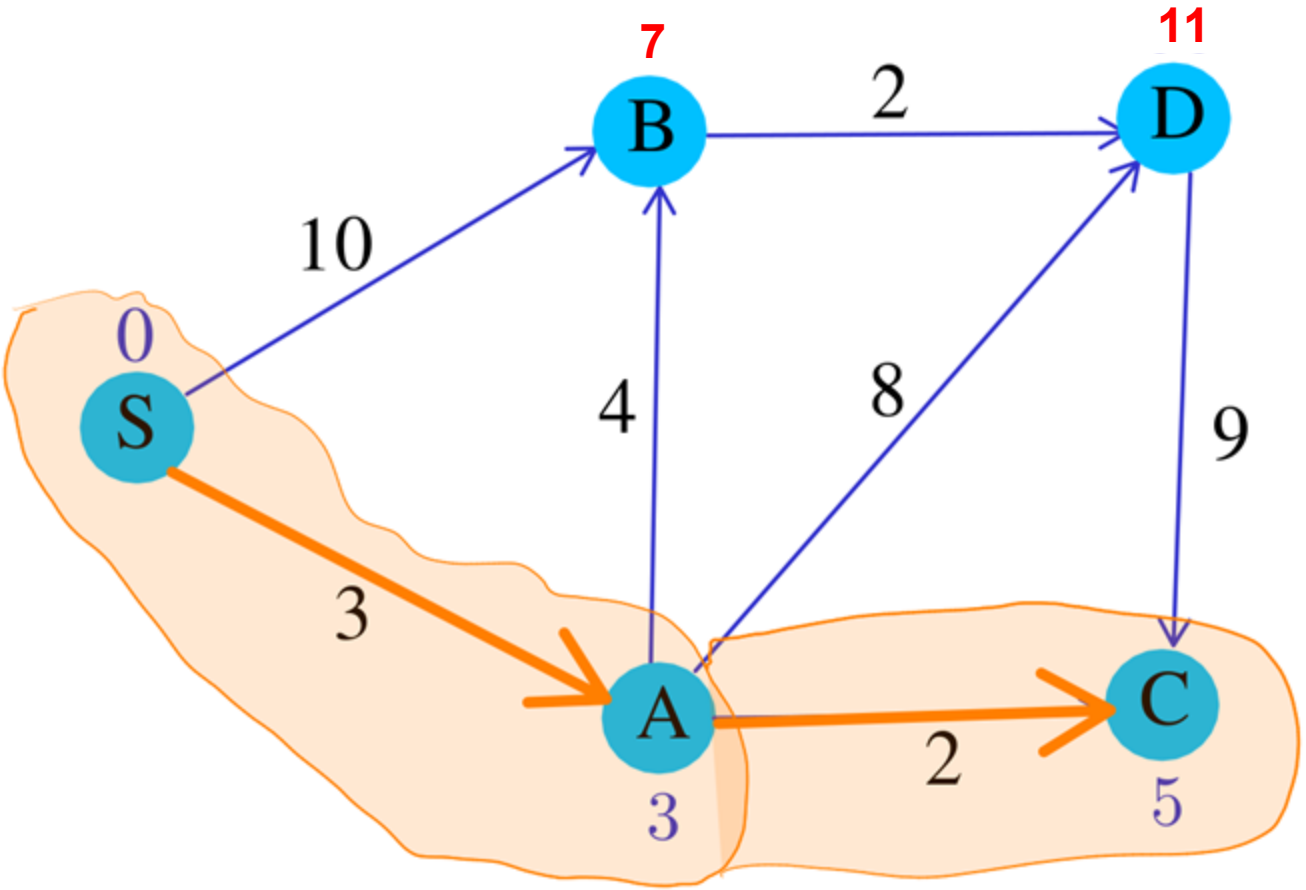
Example



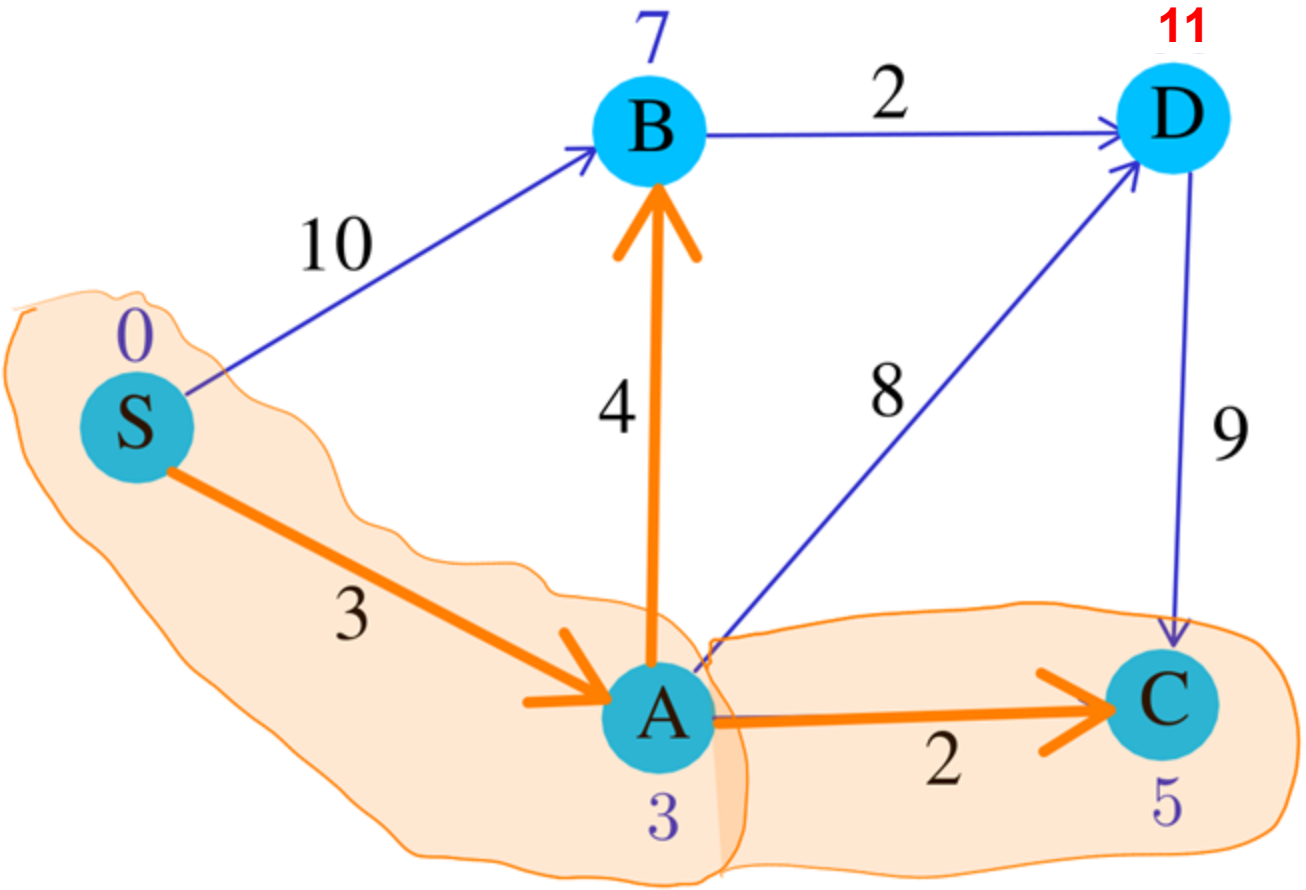
Example



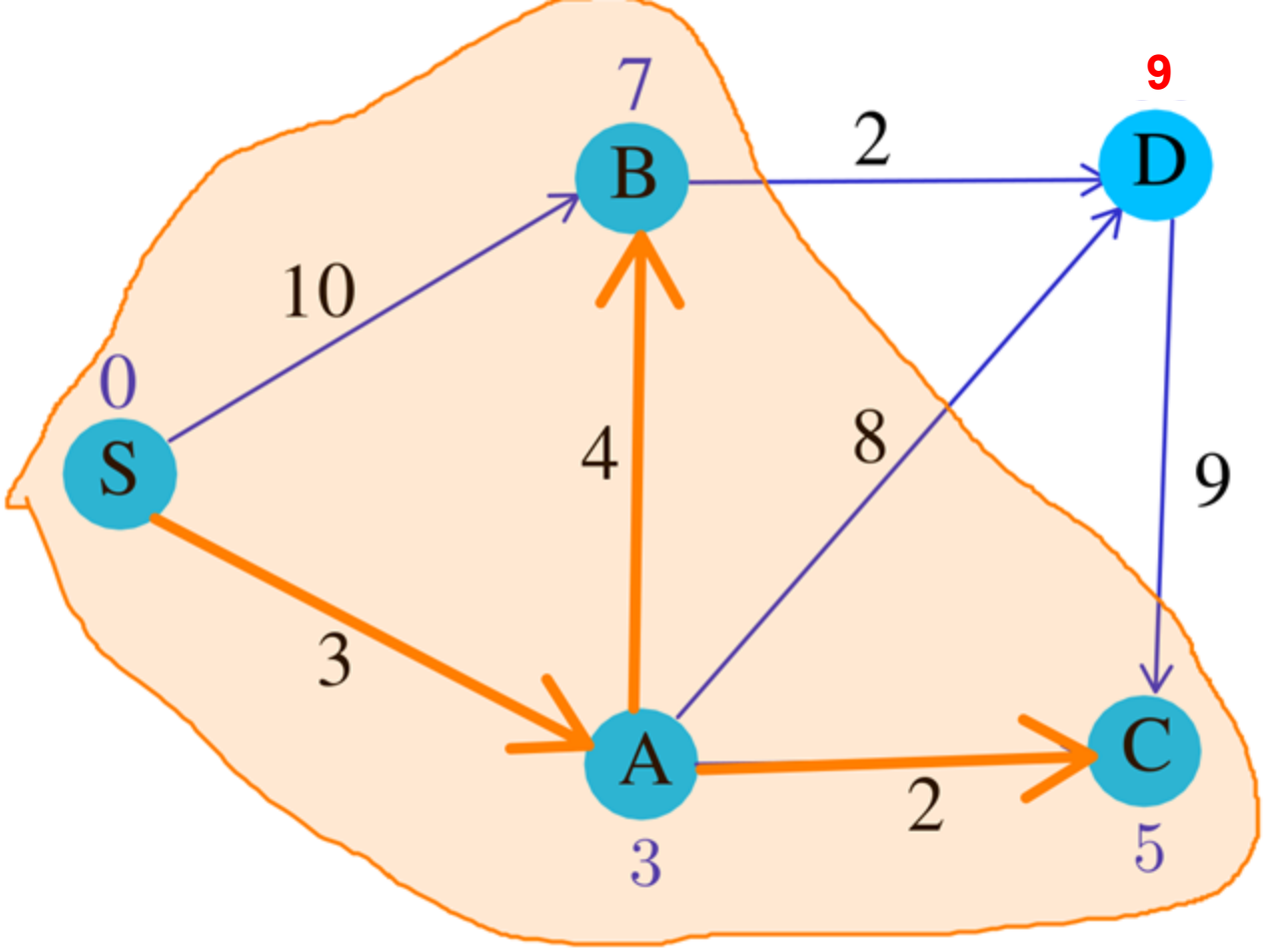
Example



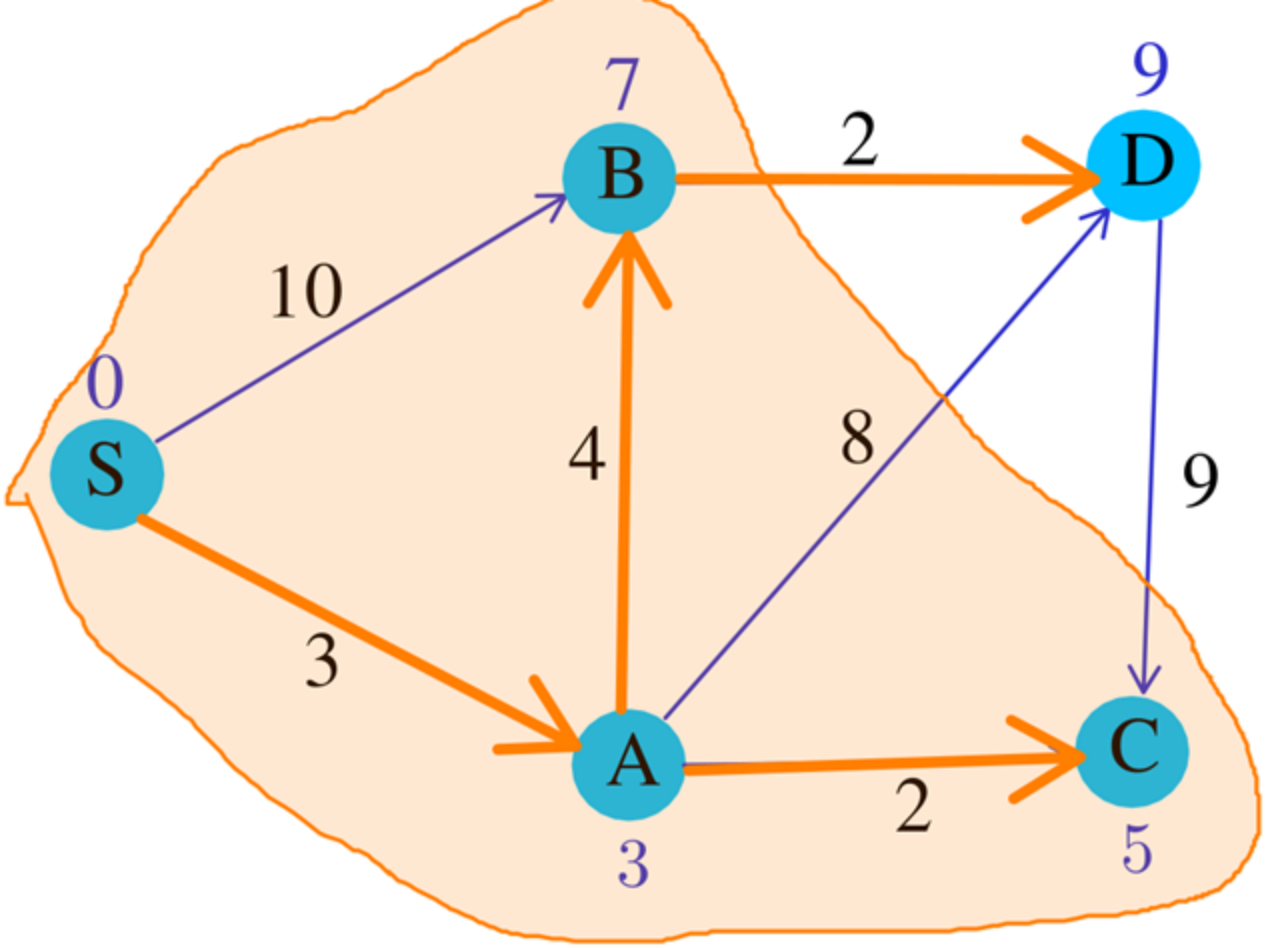
Example



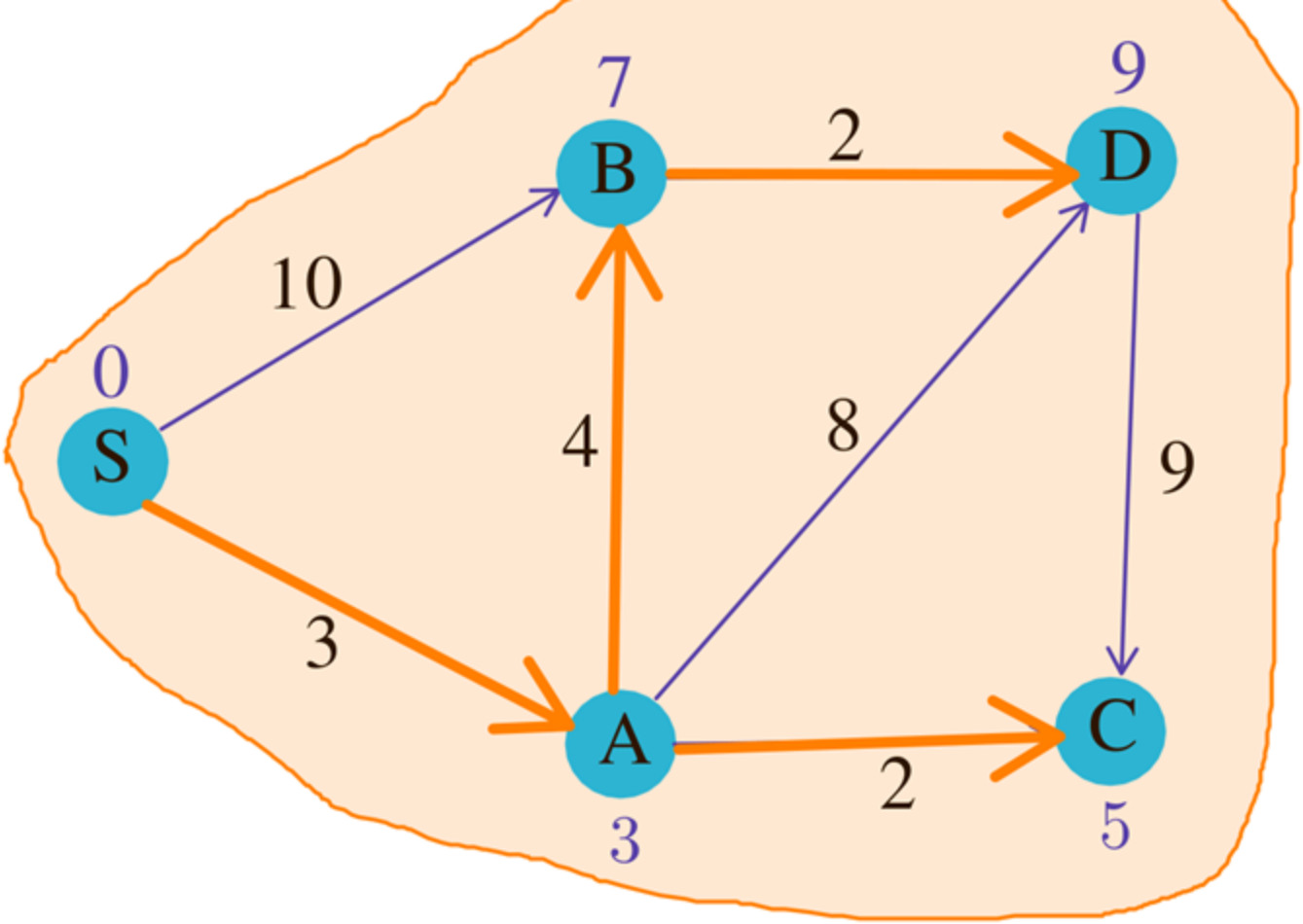
Example



Example



Example



Dijkstra's Algorithm: Implementation

```
d[s] = 0
for each v ∈ V − {s}
    d[v] = ∞
B = ∅
Q = V
while Q ≠ ∅
    u ← EXTRACT-MIN(Q)
    B ← B ∪ {u}
    for each v ∈ Adj[u]
        if (v ∉ B) and ( d[v] > d[u] + w(u, v) )
            d[v] = d[u] + w(u, v)
            parent[v] = u
```

Relaxation step

Dijkstra's Algorithm: Runtime Analysis

```
        d[s] = 0
O(V) →  for each v ∈ V − {s}
            d[v] = ∞
        B = ∅
        Q = V
O(V) →  while Q ≠ ∅
O(V) →      u ← EXTRACT-MIN(Q)
            B ← B ∪ {u}
O(deg(u)) → for each v ∈ Adj[u]
                if (v ∉ B) and ( d[v] > d[u] + w(u, v) )
                    d[v] = d[u] + w(u, v)
                    parent[v] = u
```

Relaxation step

Runtime: If the distances are stored in an array: **$O(V^2 + E) = O(V^2)$**

Dijkstra's Algorithm: Runtime Analysis

```
        d[s] = 0
O(V) →  for each v ∈ V − {s}
            d[v] = ∞
        B = ∅
        Q = V
O(V) →  while Q ≠ ∅
O(log V) →  u ← EXTRACT-MIN(Q)
            B ← B ∪ {u}
O(deg(u)) →  for each v ∈ Adj[u]
                if (v ∉ B) and ( d[v] > d[u] + w(u, v) )
                    d[v] = d[u] + w(u, v) #Decrease key of v to d[v]
O(log V)            parent[v] = u
```

Relaxation step

Q is a min-heap maintaining V–B.
The key of each node v is d[v]
Insert vertices into Q

Runtime: if the distances are stored in a priority queue(heap)

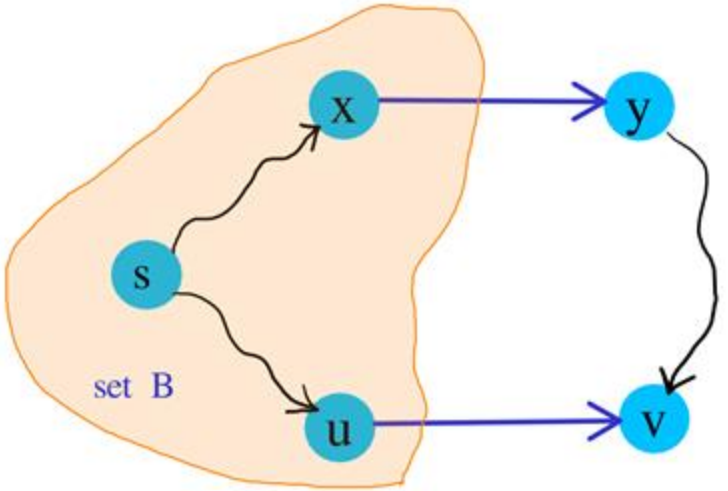
O(V log V + E log V) = O(E log V)

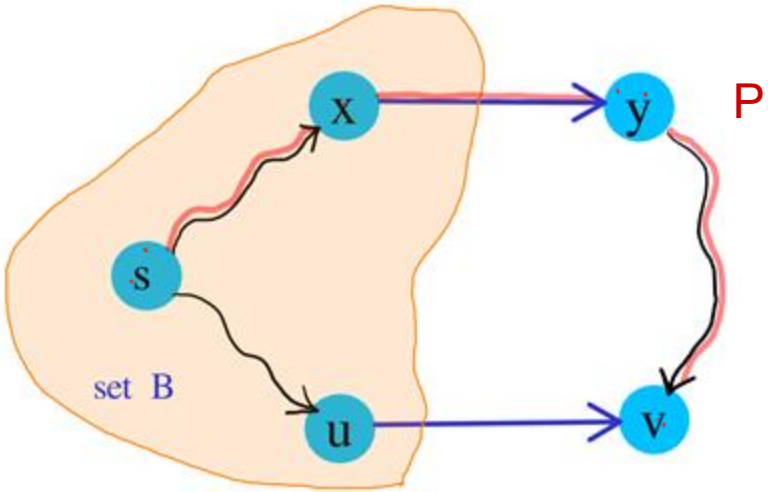
Proof of Optimality

- We prove by induction on size of of B.
- **T(k)**: |B| = k, for all u ∈ B, **d[u]** is the length of the shortest path to s for all vertices u in B
  - **Base case**: T(1) is always true. In this case B={s}, |B| = 1, and d(s) = 0
  - **Induction Hypothesis**: Suppose T(k) is true
  - **Induction Step**: Prove T(k+1) is true
    - Suppose **v** is the vertex k+1 that is added by an edge (u,v)
    - d[v] = d[u] + w(u, v) (is done by algorithm)
    - $\mathbf{P_v}$: shortest path from s to v found by algorithm ((u,v) is the final edge on s-v path P_v)
    - For contradiction, suppose P_v is not the shortest path to v, say another path P is shorter
    - This path must leave the set B somewhere. Let y be the first node on P that is not in B, and let x ∈ B be the node just before y
- w(P) ≥ w (path from s to y) = w(path from s to x) + w(x, y)
- w(path from s to x) + w(x, y) ≥ w(shortest path from s to x) + w(x, y)= d[x] + w(x,y)
- **w(P) ≥ d[x] + w(x, y) ≥ d[u] + w(u, v) = d[v]** → because of the way algorithm works
- → w(P) ≥ d[v] for any other path P from s to v

Proof of Optimality

- We prove by induction on size of of B.
- **T(k)**: |B| = k, for all u ∈ B, **d[u]** is the length of the shortest path to s for all vertices u in B

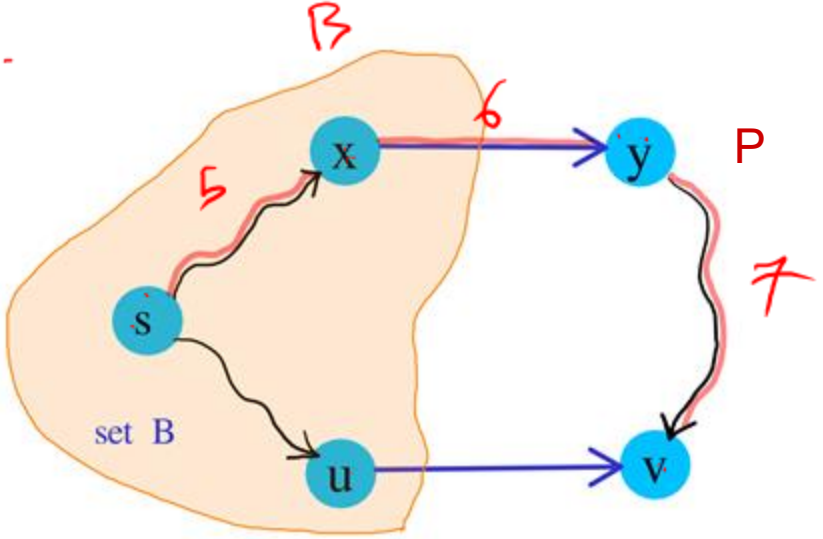




$w(P) \geq w(\text{path from s to y}) \geq d[x] + w(x, y) \geq d[u] + w(u, v) = w(P_v) = d[v]$

not true in graphs with negative weight edge

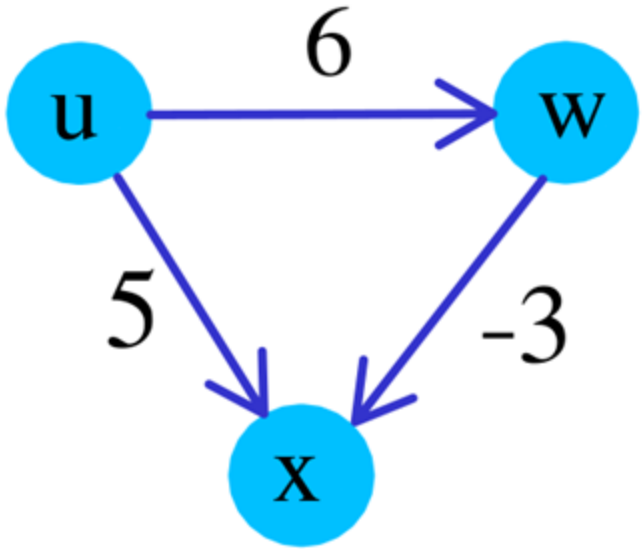
weight of a path is larger than or equal to a prefix of the path.



$$w(P) \geq w(\text{path from s to y}) \geq d[x] + w(x, y) \geq d[u] + w(u, v) = w(P_v) = d[v]$$

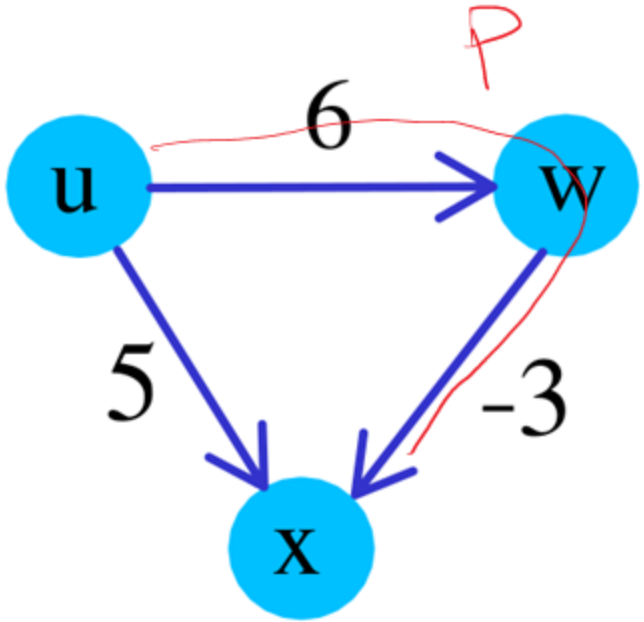
What if a graph has negative-weights edges?

Dijkstra algorithm fails



What if a graph has negative-weights edges?

Dijkstra algorithm fails



U X

U W X

6 -3

U W X

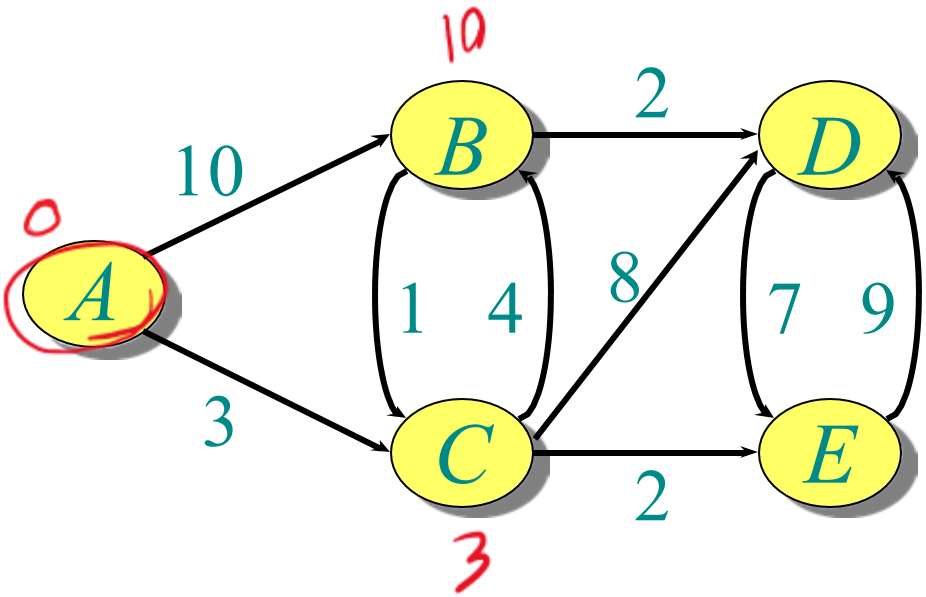
w(P) ≱ w(U W)

3 6

Example of Dijkstra's algorithm

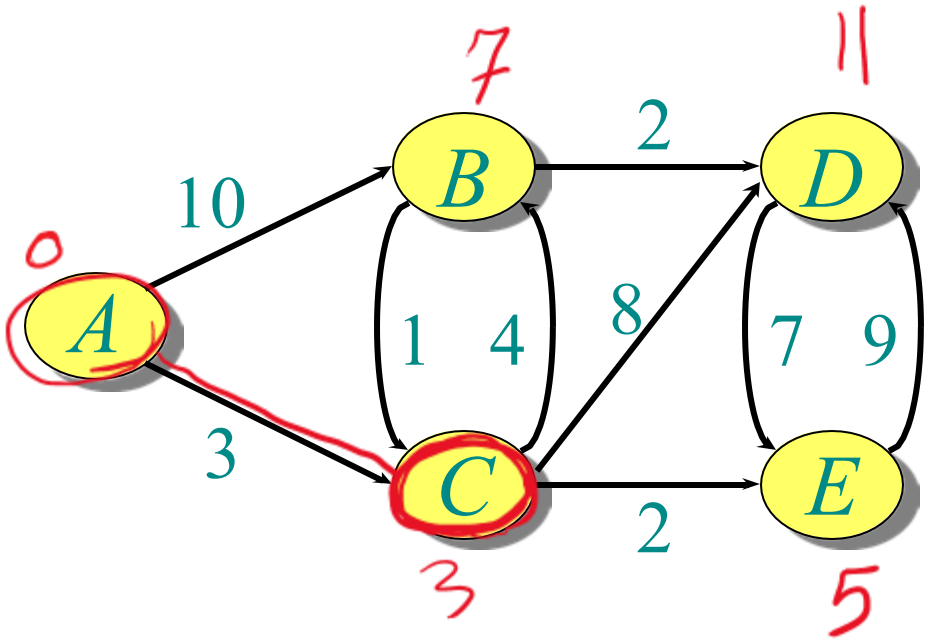
Q = A B C D E
0 ∞ ∞ ∞ ∞

0 10 3 ∞ ∞



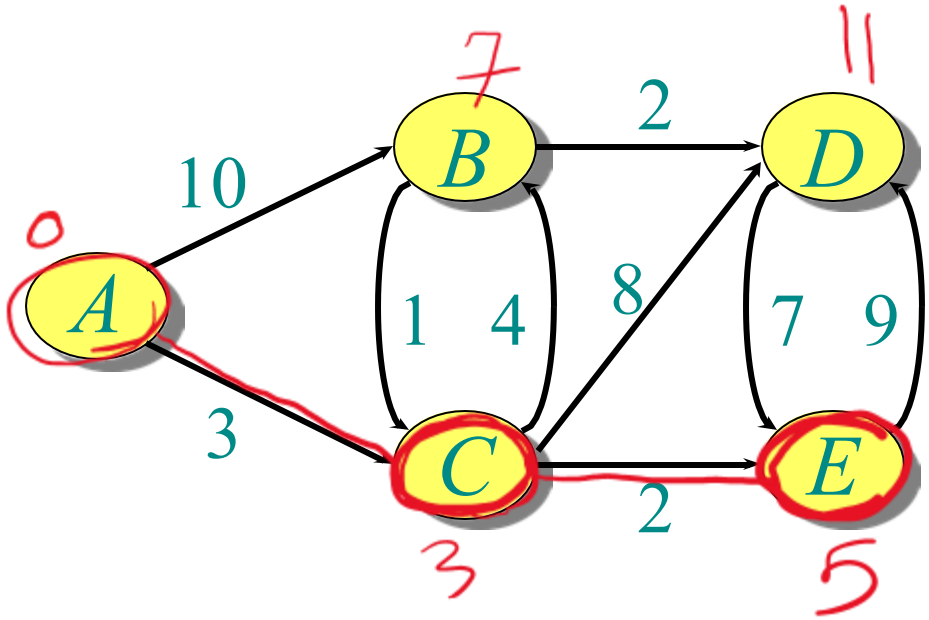
Example of Dijkstra's algorithm

Q =	A	B	C	D	E
	0	∞	∞	∞	∞
		10	3	∞	∞
		7		11	5
		7		11	

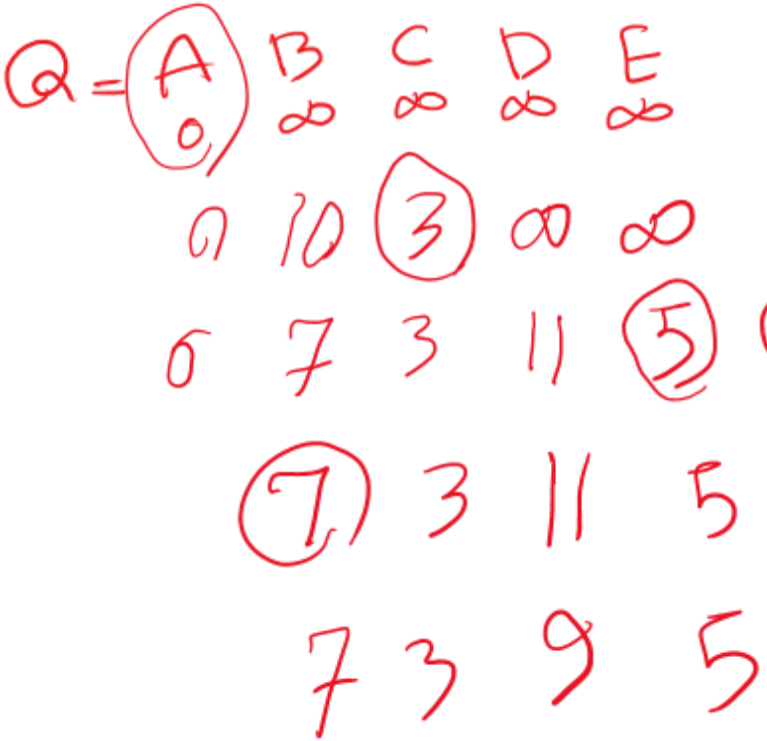


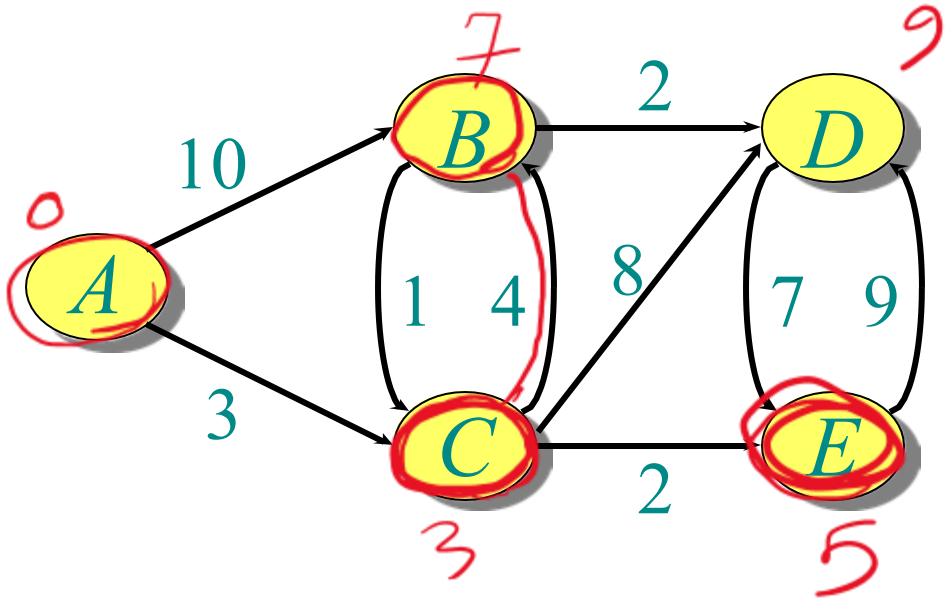
Example of Dijkstra's algorithm

Q =	A	B	C	D	E
	0	∞	∞	∞	∞
		10	3	∞	∞
		7		11	5
		7		11	5



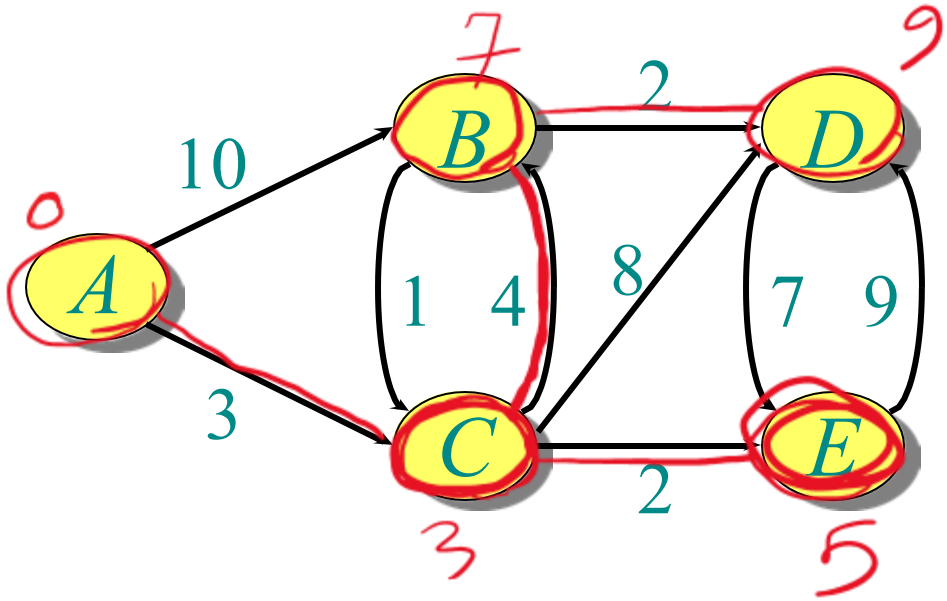
Example of Dijkstra's algorithm





Example of Dijkstra's algorithm

Q =	A	B	C	D	E
	0	∞	∞	∞	∞
	0	10	3	∞	∞
	0	7	3	11	5
	0	7	3	11	5
	0	7	3	9	5



Dijkstra: Summary

Application: routing in computer network

Pros:

- Needs non-negative edge weight
- If the weights change, we need to re-run the algorithm